四、LCEL链



1.Langchain核心

1.1 核心包的升级

前面我们使用 OpenAI SDK，是为了理解大模型最原始的调用方式。从 LCEL 开始，我们的目标不再是“调用模型”，而是“构建 AI 工作流”。

为了让 Prompt、Model、Parser、Retriever、Agent 等组件能够自由组合，我们需要使用 LangChain 的标准组件体系，因此模型部分也需要从 OpenAI SDK 升级为 ChatOpenAI。

1.1.1 Langchain-openai

（1）langchain-openai是什么

LangChain-OpenAI 是 **LangChain 生态中对 OpenAI 模型的官方封装。**它本质上是一个**可组合的模型组件**，专门设计用来在 LangChain 的工作流中作为「模型节点」使用。

- **OpenAI SDK** = 裸机发动机（功能强大，但需要自己造车身、变速箱、方向盘）
- **LangChain-OpenAI** = 已经封装好的发动机模块（可以直接插到 LangChain 这辆车上，和其他模块配合工作）

核心特点：

- 保留了原生 OpenAI 的全部能力（streaming、tool calling、structured output 等）
- 提供了与 LangChain 生态统一的输入输出接口
- 天然支持后续的 LCEL 组合、链式调用、Agent 等高级能力

（2）**与 OpenAI SDK 的核心区别**

维度	OpenAI SDK	LangChain-OpenAI	影响
定位	原始 API 客户端	LangChain 生态中的模型组件	根本差异
输入形式	严格的 messages 列表	支持字符串、ChatPromptValue、BaseMessage 等	灵活性
输出形式	原始 API Response 对象	统一的 AIMessage	一致性
可组合性	差（需要自己写大量胶水代码）	优秀（天然支持 LCEL 管道）	开发效率
生态集成	无	可直接接入 Prompt、Memory、Parser、Retriever 等	工程能力
后续扩展	困难	容易（多模型切换、Agent、复杂工作流）	长期维护

强调重点： LangChain-OpenAI **不是功能更强**，而是**可组合、可扩展、与后续技术栈兼容。**

（3）使用方法

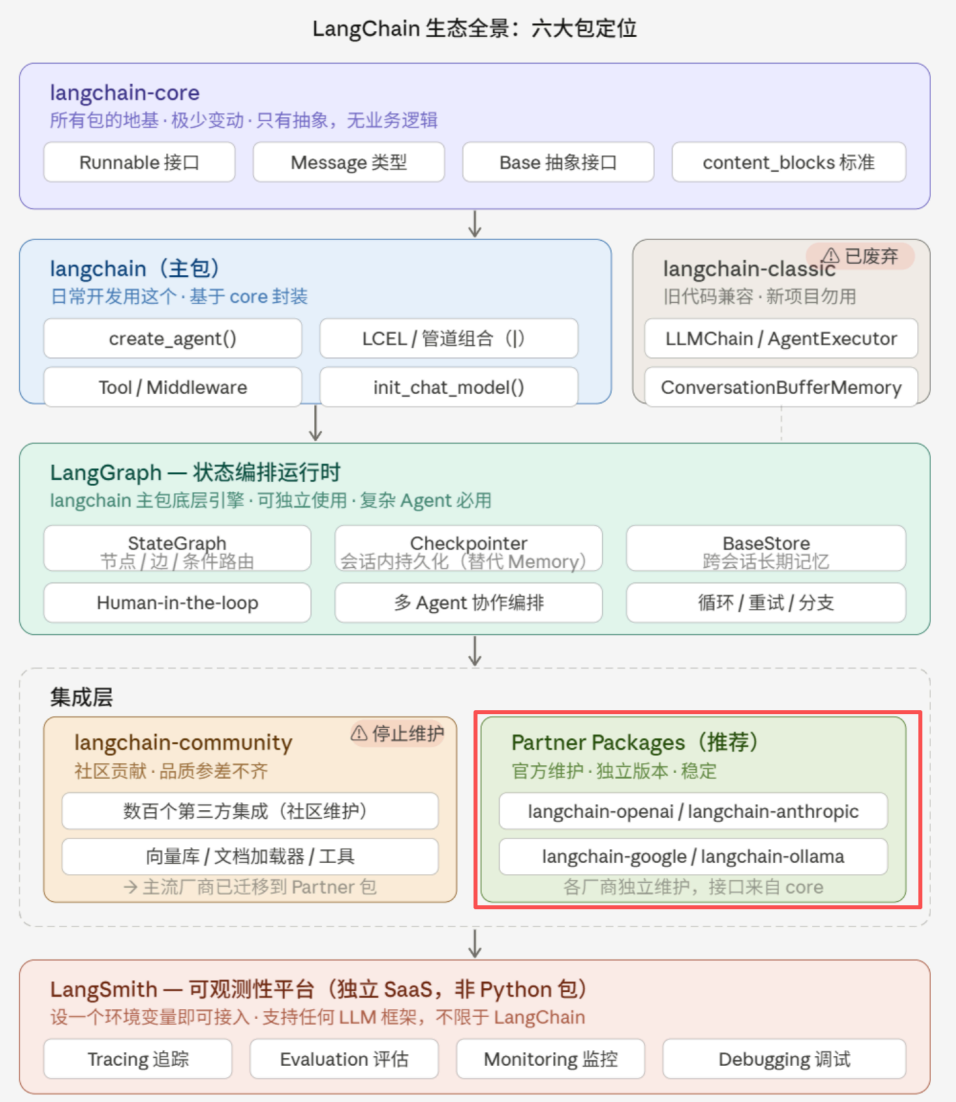
安装命令：

```
pip install langchain-openai
```

导入方式：

```python
from langchain_openai import ChatOpenAI
```

1.1.2 Partner Packages



（1）Partner Packages 是什么?

Partner Packages 是 LangChain 官方采用的一种**模块化集成机制。**

简单来说：

> LangChain 本身并不直接实现对各种模型和服务的支持，而是把这些支持拆分到一个个独立的包中，由对应的厂商或社区来维护。

这些独立的包就叫做 **Partner Packages**（伙伴包）。

核心架构可以理解为：

- **langchain-core**：LangChain 的核心，定义了 Runnable、LCEL、基础抽象等（保持轻量且稳定）
- **Partner Packages**：负责把具体的模型、服务适配到 LangChain 的统一接口上

（2）**常见表现形式**：

- `langchain-openai`

- `langchain-anthropic`
- `langchain-deepseek`
- `langchain-ollama`
- `langchain-qianfan`（百度千帆）
- `langchain-huggingface` 等

每个 Partner Package 主要负责把对应模型的 **ChatModel**、**Embeddings** 等组件实现出来，让它们能和 LangChain 其他部分无缝协作。

（3）为什么 LangChain 要设置 Partner Packages?

LangChain 采用这种设计主要有以下几个原因：

原因	具体说明
保持核心轻量	如果把所有模型的适配代码都塞进 LangChain 核心，会让核心变得非常臃肿
由厂商自己维护	模型厂商最了解自己的模型特性，由他们维护适配代码更准确、更新更及时
便于扩展和切换	想要支持新模型时，只需要安装对应的 Partner Package，而不需要修改 LangChain 核心
社区协作友好	任何厂商或开发者都可以贡献自己的 Partner Package
统一接口	无论使用哪个模型，开发者面对的接口基本一致

总结一句话：

Partner Packages 机制让 **LangChain 既保持了核心的简洁稳定，又拥有了极强的扩展能力。**

1.1.3 常见的 Partner Packages 有哪些?

Partner Packages 主要分为以下几类（按使用频率和课程相关性排序）：

（1）LLM（大语言模型）类（最常用）

- `langchain-openai`（OpenAI GPT 系列）
- `langchain-anthropic`（Claude 系列）
- `langchain-deepseek`（DeepSeek 系列）
- `langchain-ollama`（本地模型）
- `langchain-qianfan`（百度千帆）
- `langchain-dashscope`（阿里通义千问）

（2）Embeddings（嵌入模型）类

- `langchain-openai`（同时包含 Embedding）
- `langchain-huggingface`
- `langchain-ollama`

（3）Vector Stores（向量数据库）类

- `langchain-chroma`

- `langchain-faiss`
- `langchain-milvus`
- `langchain-pinecone`
- `langchain-weaviate`

1.2 优化升级

在前面的学习中，我们主要使用原生 openai 包 + 直接在代码中写配置的方式来完成 Model I/O 和简单 RAG 的功能。这种方式虽然能快速上手，但存在以下明显问题：

- 配置信息（API Key、Base URL 等）硬编码在代码中，**安全性差且难以维护**
- 项目依赖管理较为混乱，缺少规范的依赖声明
- 代码结构相对松散，不利于后续扩展复杂 RAG 和 Agent 功能
- 与 LangChain 生态的结合不够自然

因此，我们现在进行一次**整体升级**，目标是让代码从“能跑通”进化到“可维护、可扩展、符合工程规范”。

1.2.1 环境变量管理

在初级阶段，我们把API_KEY直接放在文件里面，这样的操作相对并不安全，将 API 密钥存储在环境变量中是最佳实践，这样可以避免在代码中硬编码敏感信息。

1.创建环境变量文件 (.env)

在项目根目录创建 `.env` 文件，添加 API 密钥：

```
# .env 文件
OPENAI_API_KEY = "your_api_key"
OPENAI_API_BASE = "https://api.deepseek.com"
```

2.安装必要的依赖

```
pip install python-dotenv
```

3.添加环境变量加载：

```
# 在文件开头添加
from dotenv import load_dotenv
load_dotenv()  # 加载 .env 文件中的环境变量

# 现在可以安全地导入 LangChain 组件
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
# ... 其他导入 ...
```

4.修改模型初始化方式

不需要在代码中直接传递 API 密钥，LangChain 会自动从环境变量读取：

```
# 修改前（硬编码密钥 - 不安全！）
# storyteller_llm = ChatOpenAI(api_key="sk-...", model="deepseek_chat")

# 修改后（从环境变量读取）
# 方法一：默认（设置成OPENAI_API_KEY）系统自动读取
llm = ChatOpenAI(model = 'deepseek_chat')

# 方法二：设置成其他的名字
import os
storyteller_llm = ChatOpenAI(model="deepseek_chat",api_key =
os.getenv("DEEPSEEK_API_KEY"),base_url=os.getenv("DEEPSEEK_API_BASE"))
```

1.2.2 OpenAI的对话模块

```
from openai import OpenAI
# 调用聊天
client = OpenAI()
response1 = client.chat.completions.create(
    model = "deepseek-chat",
    messages = [{"role": "user", "content": "Hello"}],
    stream = False
)
print(response1.choices[0].message.content)
```

生成结果：

```
ChatCompletion(id='23682f5d-eeb6-4f9a-92b2-adac6f8b73ca', choices=
[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='Hello! 😊 How can I assist you today?',
refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None))],
created=1750413972, model='deepseek-chat', object='chat.completion', service_tier=None,
system_fingerprint='fp_8802369eaa_prod0425fp8',
usage=CompletionUsage(completion_tokens=11, prompt_tokens=4, total_tokens=15,
completion_tokens_details=None,
prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cached_tokens=0),
prompt_cache_hit_tokens=0, prompt_cache_miss_tokens=4))
```

1.2.3 解析：

从 DeepSeek API 返回的这个 `ChatCompletion` 对象中，可以提取以下有效信息：

1.基础信息

字段	值	说明
对话ID	`23682f5d-eeb6-4f9a-92b2-adac6f8b73ca`	本次对话的唯一标识符
模型名称	`deepseek-chat`	使用的模型是 DeepSeek 的通用聊天模型
对象类型	`chat.completion`	返回的是聊天完成对象
创建时间	`1750413972`	Unix 时间戳（对应 2025-06-20 11:26:12 UTC）

2.回复内容

字段	值	说明
回复文本	`Hello! 😊 How can I assist you today?`	AI 生成的回复内容
回复角色	`assistant`	消息来自助手
结束原因	`stop`	正常结束（非截断或长度限制）

3.Token 使用统计

统计项	值	说明
提示词 Token	`4`	用户输入的 Token 数
生成 Token	`11`	AI 回复的 Token 数
总 Token	`15`	本次交互总消耗 Token
提示缓存命中 Token	`0`	无缓存命中
提示缓存未命中 Token	`4`	全部提示词未命中缓存

1.2.4 使用langchain_openai的对话模型

```
from langchain_openai import ChatOpenAI
client = ChatOpenAI(model = "deepseek-chat")
response2 = client.invoke("Hello")
print(response2)
```

生成结果：

```
content='Hello! 😊 How can I assist you today?' additional_kwargs={'refusal': None}
response_metadata={'token_usage': {'completion_tokens': 11, 'prompt_tokens': 4,
'total_tokens': 15, 'completion_tokens_details': None, 'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0,
'prompt_cache_miss_tokens': 4}, 'model_name': 'deepseek-chat', 'system_fingerprint':
'fp_8802369eaa_prod0425fp8', 'finish_reason': 'stop', 'logprobs': None} id='run-
443d034a-e8b5-4633-aa7f-cf614cbf02cf-0' usage_metadata={'input_tokens': 4,
'output_tokens': 11, 'total_tokens': 15, 'input_token_details': {'cache_read': 0},
'output_token_details': {}}
```

1.2.5 解析

1.基础信息

信息类别	值	说明
模型生成内容	`'Hello! 😊 How can I assist you today?'`	AI 生成的实际回复文本
拒绝状态	`None`	未检测到拒绝回答（正常响应）
模型名称	`'deepseek-chat'`	使用的模型标识
完成原因	`'stop'`	正常结束（非截断或长度限制）
系统指纹	`'fp_8802369eaa_prod0425fp8'`	服务端配置标识

2.Tokens 使用详情

统计项	值	说明
提示 Token	`4`	用户输入消耗的 Token 数
生成 Token	`11`	AI 回复消耗的 Token 数
总 Token	`15`	本次交互总消耗 Token
提示缓存命中	`0`	缓存命中的 Token 数
提示缓存未命中	`4`	缓存未命中的 Token 数
缓存读取	`0`	从缓存读取的 Token 数

3.调试与追踪信息

信息类别	值	说明
运行ID	`'run-443d034a-e8b5-4633-aa7f-cf614cbf02cf-0'`	LangChain 内部执行标识
概率信息	`None`	未返回 token 概率数据
Token 详情	`None`	未返回生成 token 详情

信息类别	值	说明
Token 详情	`None`	无音频 token 使用
Token 详情	`0`	缓存 token 数量

1.2.6 两者对比

对比维度	原生 OpenAI SDK (openai)	LangChain-OpenAI (langchain_openai)	优势方
返回对象类型	ChatCompletion（嵌套较深）	AIMessage（结构更扁平）	LangChain
获取回复内容	response.choices[0].message.content	response.content	**LangChain**
代码简洁度	一般（需要写较长路径）	更好（直接属性访问）	**LangChain**
与 LangChain 集成	几乎无法直接集成	天然支持（可直接用于 LCEL、Chain、Agent）	**LangChain**
可扩展性	较差	强（后续可轻松接入 Memory、Parser、Tools 等）	**LangChain**

通过上面两个示例我们可以看到：

- 原生 SDK 虽然功能强大，但返回结构比较复杂，提取信息时需要写较长的路径。
- 而 langchain_openai 返回的 AIMessage 对象更简洁，核心内容可以直接通过 .content 获取。
- 更重要的是，LangChain-OpenAI 是为后续的 **LCEL、复杂 RAG、Agent** 设计的，它能更好地和其他组件组合使用。

1.2.7 多模型 API 支持

在前面的学习中，我们主要使用 DeepSeek 模型进行演示。随着项目复杂度提升，我们往往需要根据不同场景选择更合适的模型。

在 LangChain 中，支持多模型切换主要有两种方式，而目前**最推荐、最稳定的方式**是使用 **COpenAI 兼容接口。**

（1）统一接口优势

目前国内主流大模型厂商（DeepSeek、百度千帆、硅基流动、Moonshot、智谱等）大多都提供了 **OpenAI 兼容的 API 接口。**这种方式有以下优势：

- 代码改动极小，只需要修改 `model`、`api_key` 和 `base_url`
- 学习成本低，一套代码可以适配多个模型
- 与 LangChain 生态结合良好
- 避免因为厂商接口差异导致的频繁代码修改

（2）统一接入方式（推荐写法）

只要你拿到了对应模型的 `API_KEY` 和 `BASE_URL`，就可以使用以下方式进行调用：

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="deepseek-chat",                    # 替换为目标模型名称
    openai_api_key="sk-xxxxxxxxxxxxxxxx",     # 对应厂商的 API Key
    openai_api_base="https://api.deepseek.com" # 对应厂商的 Base URL
)

response = llm.invoke("你好，请简单介绍一下自己")
print(response.content)
```

核心要点：

- `model`：填写目标模型的名称
- `openai_api_key`：填写对应厂商提供的 Key
- `openai_api_base`：填写对应厂商的 API 地址（Base URL）

这种写法在目前国内大多数模型上都可以直接使用。

1.2.8 Embeddings 封装升级

在前面的简单 RAG 项目中，我们使用的是 `transformers` 库直接加载 embedding 模型来生成向量。这种方式虽然可以快速实现功能，但在实际工程中使用时会遇到以下问题：

（1）两者对比

问题维度	使用原生 `transformers` 的痛点	LangChain Embeddings 的优势
接口一致性	需要自己写 embedding 函数，和 LLM 调用风格不统一	与 `ChatOpenAI` 保持一致的 Runnable 接口
模型切换成本	换 embedding 模型需要大幅修改代码	只需更换类即可切换模型
与 RAG 集成	需要手动把 embedding 结果和检索逻辑拼起来	可直接与 `VectorStore`、`Retriever` 配合使用
后续扩展性	难以接入重排序（Reranker）、混合检索等高级功能	为高级 RAG 和 LCEL 提供良好基础

总结：

原生 `transformers` 方式更适合做**实验和验证**，而 LangChain 的 Embeddings 抽象更适合做**可维护、可扩展的生产级 RAG。**

（2）LangChain 中的 Embeddings 抽象

LangChain 把 Embedding 模型也抽象成了统一的接口，主要包含两个核心方法：

- `embed_query(text)`：对单个查询文本进行 embedding（常用于用户问题）
- `embed_documents(texts)`：对多个文档进行批量 embedding（常用于知识库）

这种设计让 embedding 模块可以像 `ChatOpenAI` 一样，作为 LangChain 工作流中的一个标准组件使用。

（3）升级后的推荐写法

使用 LangChain 封装后的 Embeddings 示例：

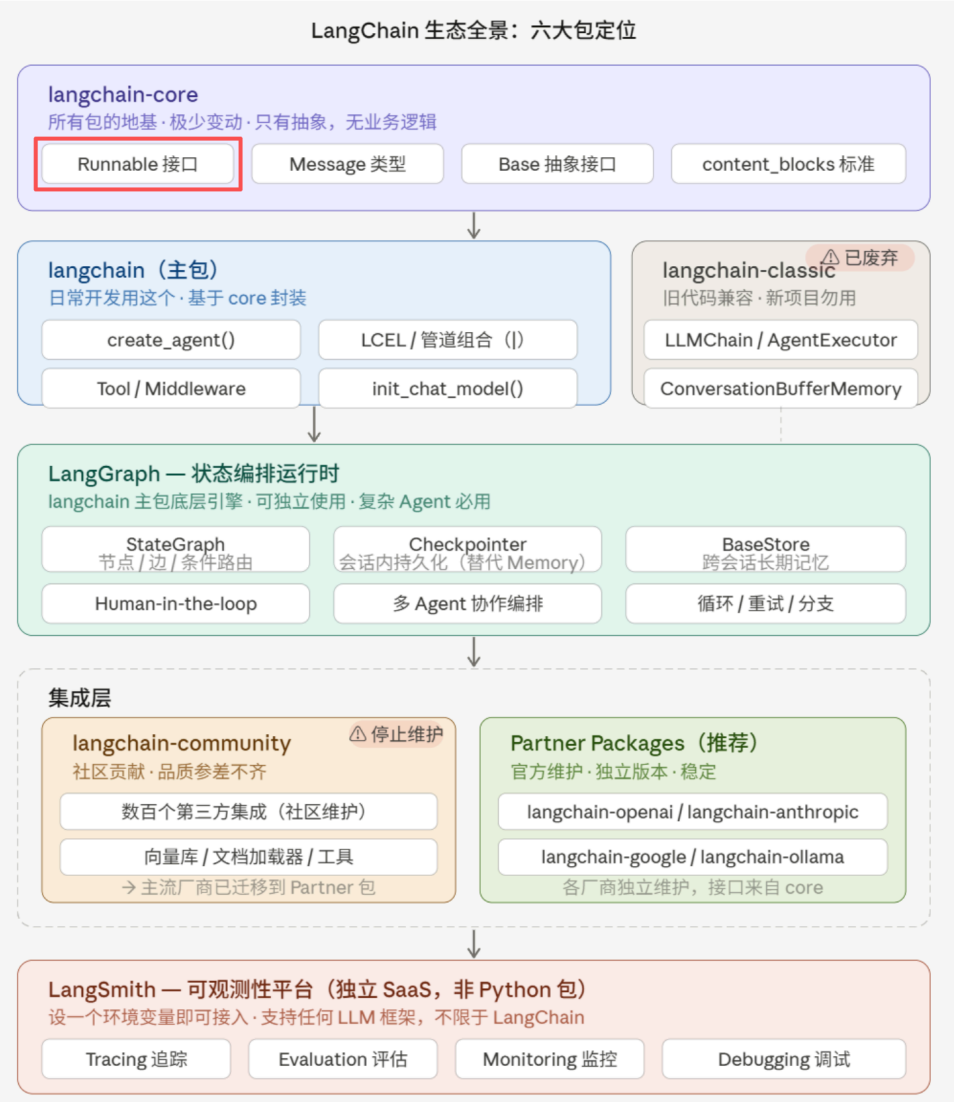
```python
from langchain_openai import OpenAIEmbeddings

# 使用 OpenAI 兼容的 Embedding 模型
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small",           # 或其他支持的 embedding 模型
    openai_api_key="sk-xxxx",
    openai_api_base="https://api.deepseek.com"  # 根据实际厂商调整
)

# 对单个 query 进行 embedding
query_embedding = embeddings.embed_query("什么是 RAG？")

# 对多个文档进行批量 embedding
docs = ["文档1内容", "文档2内容", "文档3内容"]
doc_embeddings = embeddings.embed_documents(docs)
```

2.Runnable协议



在完成前面的工程化升级后，我们已经理解了为什么要从原生 OpenAI SDK 转向 LangChain。那么接下来，我们需要解决一个更核心的问题：

LangChain 是如何把各种组件（LLM、Prompt、Parser、Retriever 等）组织起来，并让它们能够灵活组合的?

答案就是 **Runnable 协议。**

2.1 概述

2.1.1 什么是Runnable协议

Runnable 是 LangChain 中所有可执行组件所遵循的**统一接口规范**（Interface）。

简单来说，Runnable 是一个**抽象基类**，它定义了组件应该具备的标准行为。LangChain 中几乎所有的核心组件都实现了这个接口，包括：

- `ChatOpenAI`、`OpenAI` 等语言模型
- `PromptTemplate`、`ChatPromptTemplate` 等提示模板
- `StrOutputParser`、`JsonOutputParser` 等输出解析器

- `Retriever`、`VectorStoreRetriever` 等检索器
- 甚至包括自定义的函数和链

只要一个组件实现了 `Runnable` 接口，它就可以被统一地调用、组合和扩展。Runnable 本身并不实现具体功能，它更像是一套**契约**，规定了所有组件应该支持哪些标准操作。

2.1.2 发展历程

Runnable 协议的出现并不是一开始就有的，它是 LangChain 在实践中不断演进的结果。

早期阶段（Chain 时代）：

- LangChain 最初主要通过继承 `Chain` 类来构建复杂流程。
- 开发者需要通过 `._call()` 方法来实现具体逻辑。
- 这种方式的问题是：**组合性差、流式支持弱、代码侵入性强**，难以灵活地拼接不同组件。

过渡阶段：

- 随着 LLM 应用越来越复杂，早期 Chain 的局限性逐渐暴露。
- 开发者需要更灵活的组合方式、更好的流式支持，以及更低的组件耦合度。

Runnable 时代（2023 年底至今）：

- LangChain 引入了 `Runnable` 协议，彻底改变了组件的组织方式。
- 所有组件不再依赖继承，而是通过实现统一接口来获得组合能力。
- 随后推出了 **LCEL（LangChain Expression Language）**，允许开发者使用管道符 `|` 以声明式的方式组合 Runnable 组件。
- 这一转变使得 LangChain 从“面向对象的链式调用”转向了“函数式 + 声明式的组合范式”。

当前状态：

- Runnable 已成为 LangChain 的核心抽象。
- LCEL、LangGraph 等上层能力都构建在 Runnable 协议之上。
- 它既保留了高度的灵活性，又提供了良好的开发体验和可观测性。

2.2 Runnable 的核心结构

Runnable 并不是一个复杂的概念，它本质上是一套**让组件能够被统一调用和组合的结构规范**。下面我们从两个层面来理解它的核心结构。

2.2.1 核心设计理念

Runnable 的设计主要围绕以下三个核心理念展开：

（1）统一接口原则

LangChain 中所有可执行的组件（LLM、Prompt、Parser、Retriever 等），都必须实现同一套方法。

这意味着：

- 无论你面对的是 ChatOpenAI 还是 StrOutputParser，**调用方式都是相似的。**
- 开发者不需要为不同组件写适配层代码。
- 所有组件都可以被当作“标准零件”进行自由组合。

这种统一接口是实现组件可插拔、可替换的基础。

（2）声明式编程范式

Runnable 鼓励使用**声明式**的方式来构建工作流，而不是通过写大量过程式代码来拼接。

最典型的表现就是使用管道操作符 来连接组件：

```
chain = prompt | llm | parser
```

这种写法就像“拼装乐高积木”一样直观：

- 左侧组件的输出会自动成为右侧组件的输入。
- 开发者只需要描述“想要什么”，而不需要关心中间具体的调用和转换逻辑。

这种声明式风格大大降低了构建复杂链的难度，也让代码更易读、易维护。

（3） 数据流抽象

Runnable 将组件之间的交互抽象为清晰的**数据流**：

输入 → 处理 → 输出

在这个过程中，Runnable 负责：

- 自动进行类型转换（比如把字符串转成消息列表）
- 传递配置信息（tags、metadata 等）
- 错误处理和传播
- 支持流式输出

这种数据流抽象让开发者可以专注于业务逻辑，而不是底层的数据传递和转换细节。

2.2.2 Runnable 的核心方法

Runnable 定义了一组标准方法，这些方法是所有组件必须支持的。理解这些方法，是掌握 Runnable 的关键。Runnable 的核心方法主要分为以下几类：

方法	类型	作用	常用场景
invoke	同步	执行一次调用，输入单个数据	普通对话、单次推理
batch	同步	批量执行，输入多个数据	批量 embedding、批量生成
stream	同步	流式输出，逐步返回结果	实时对话、长文本生成
ainvoke	异步	异步版本的 invoke	异步 Agent、高并发场景
astream	异步	异步版本的 stream	异步流式对话

教学重点说明：

- invoke 是最基础、最常用的方法，理解它就能理解 Runnable 的基本工作方式。
- stream 是现代 LLM 应用中非常重要的能力，Runnable 对它的原生支持是其一大优势。
- 异步方法（a 开头）为构建高性能应用提供了支持。
- 这些方法在所有 Runnable 组件上行为一致，这是 Runnable 统一接口的体现。

2.3 Runnable 的输入与输出

Runnable 协议的一个重要特点是**输入和输出的标准化**。虽然不同组件的功能差异很大，但 LangChain 通过 Runnable 接口对它们的输入输出做了统一抽象，这使得组件之间能够顺畅地组合。

2.3.1 主要组件的输入输出类型

下表展示了 LangChain 中常见 Runnable 组件的**主要输入和输出类型**：

组件类型	常用类	主要输入类型	主要输出类型	说明
语言模型	`ChatOpenAI`	`str` 或 `PromptValue`	`AIMessage`	最核心的组件
提示模板	`ChatPromptTemplate`	`dict`	`PromptValue`	用于格式化提示
输出解析器	`StrOutputParser`	`AIMessage`	`str`	将模型输出转为字符串
检索器	`Retriever`	`str`	`List[Document]`	知识库检索
自定义逻辑	`RunnableLambda`	`Any`	`Any`	用于自定义处理

说明：

- 大部分情况下，我们不需要关心底层复杂的类型结构。
- LangChain 会自动帮我们进行类型转换，让组件能够顺利衔接。

2.3.3 提示词类型：`PromptValue`

`PromptValue` 是 LangChain 对“发送给模型的提示”的统一表示。它主要有两种形式：

- **字符串形式**（`StringPromptValue`）
- **消息列表形式**（`ChatPromptValue`）

示例

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage

# 创建一个聊天提示模板
```

```python
chat_prompt = ChatPromptTemplate.from_messages([
    SystemMessage(content="你是一位{role}专家"),
    HumanMessage(content="请用一句话解释{concept}")
])

# 通过 invoke 生成 PromptValue
prompt_value = chat_prompt.invoke({
    "role": "量子物理",
    "concept": "量子纠缠"
})

print(type(prompt_value))
print(prompt_value.to_string())       # 转为字符串
print(prompt_value.to_messages())     # 转为消息列表
```

重点理解：

- PromptValue 是连接"提示模板"和"语言模型"的桥梁。
- 它提供了 to_string() 和 to_messages() 两个方法，方便在不同场景下使用。

2.3.4 模型输出类型：`AIMessage`

AIMessage 是语言模型返回的最主要结果类型，几乎所有 LLM 的输出最终都会被包装成 AIMessage。

示例：

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="deepseek-chat")

response = llm.invoke("请用一句话介绍 LangChain")

print(type(response))           # <class 'langchain_core.messages.ai.AIMessage'>
print(response.content)          # 模型生成的主要内容
print(response.response_metadata)  # 包含 finish_reason、model_name 等信息
print(response.usage_metadata)     # token 使用情况
```

重点理解：

- 实际开发中，我们**最常用**的就是 response.content 来获取模型的回复。
- response_metadata 和 usage_metadata 包含了模型返回的元信息（如 token 消耗、结束原因等），在需要做监控或计费时会用到。

2.3.5 输出解析器（Output Parser）的价值

当我们使用 LangChain 时，**模型返回的并不是纯字符串，而是 AIMessage 对象。**如果我们希望直接拿到干净的字符串，就需要使用输出解析器。

示例对比：

```python
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model="deepseek-chat")

# 不使用输出解析器
response = llm.invoke("请用一句话介绍 LangChain")
print(response.content)            # 能拿到内容，但类型是 AIMessage

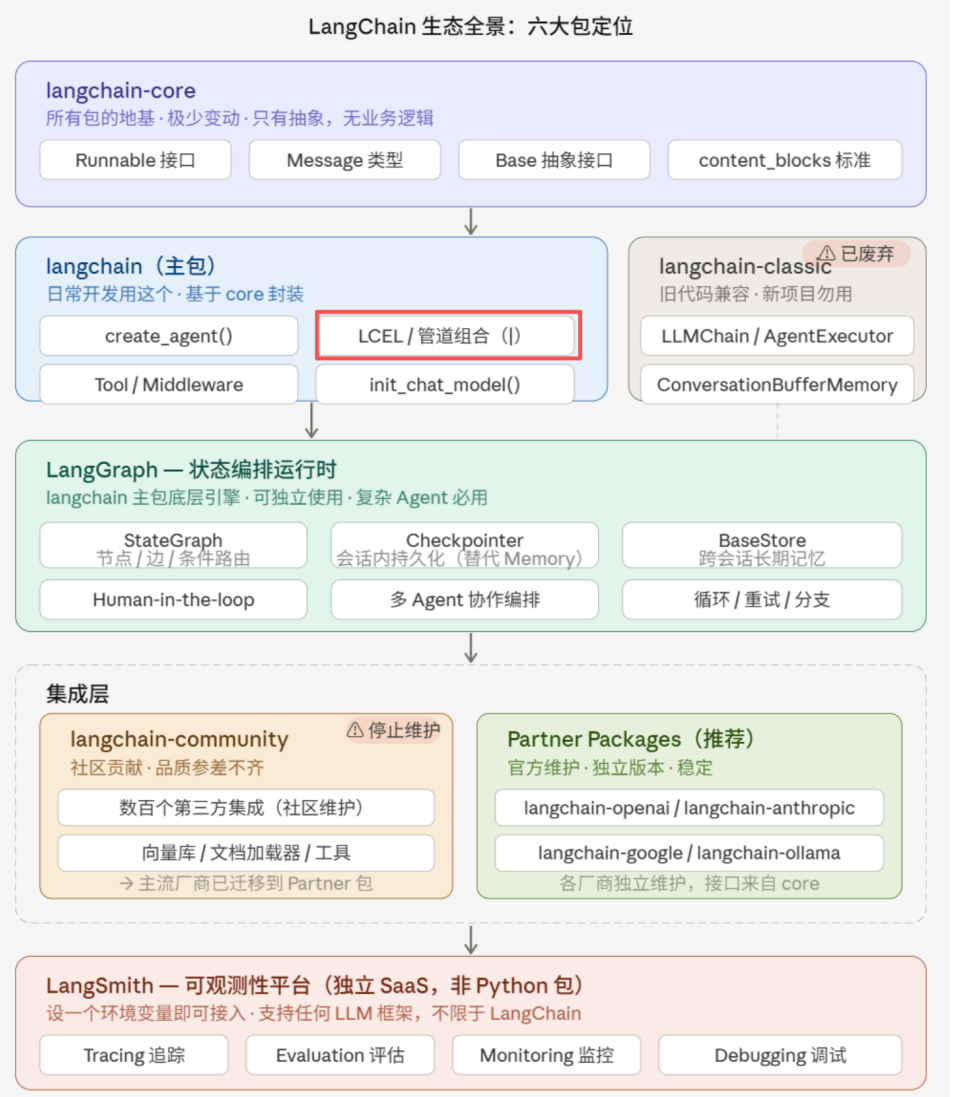
# 使用输出解析器
parser = StrOutputParser()
chain = llm | parser               # 使用 LCEL 组合

result = chain.invoke("请用一句话介绍 LangChain")
print(result)                      # 直接拿到字符串
print(type(result))                # <class 'str'>
```

重点理解：

- StrOutputParser 的核心作用就是把 AIMessage 转换为纯字符串，让输出更简洁、统一。
- 除了 StrOutputParser 之外，LangChain 还提供了 JsonOutputParser、PydanticOutputParser 等其他解析器，用于将模型输出转换为 JSON 格式或特定数据结构，满足结构化输出的需求。

2.4 LCEL



在学习了 Runnable 协议之后，我们终于可以接触 LangChain 中最核心、最实用的能力之一 —— **LCEL**（LangChain Expression Language）。

2.4.1 什么是 LCEL?

LCEL 是 LangChain 提供的一种**声明式编程语言**，用于以简洁、优雅的方式组合多个 Runnable 组件。

它的核心特征是使用**管道操作符** `|` 来连接不同的组件，就像在命令行中使用管道一样：

```
chain = prompt | llm | parser
```

这种写法具有以下优势：

- **代码简洁**：不需要手动编写大量胶水代码
- **可读性强**：从左到右清晰地展示了数据流向
- **类型安全**：LangChain 会自动处理组件之间的输入输出转换
- **支持流式、并行、回退等高级能力**

简单来说，**LCEL 就是 Runnable 协议的“组合语言”**。它让开发者能够像搭积木一样快速构建复杂的 LLM 应用。

2.4.2 为什么需要 LCEL?

（1）传统的CHAIN

在 LCEL 出现之前，LangChain 主要使用继承 Chain 的方式来构建流程。那时的写法通常是这样的：

```
# 传统 Chain 的写法（较繁琐）
from langchain.chains import LLMChain

chain = LLMChain(prompt=prompt, llm=llm)
result = chain.run({"concept": "量子纠缠"})
```

这种方式存在几个明显问题：

- 代码不够灵活，扩展和修改比较麻烦
- 流式输出、并行执行等能力支持较弱
- 不同组件之间需要手动处理数据传递

（2）LCEL chain

而 **LCEL** 的出现，彻底改变了这种局面。它允许我们使用更现代、更简洁的方式来组合组件：

```
# 使用 LCEL 的写法
chain = prompt | llm | parser
result = chain.invoke({"concept": "量子纠缠"})
```

LCEL 的核心优势：

- **代码更简洁**：使用管道符 就能完成组件组合
- **可读性更强**：从左到右清晰展示数据流
- **扩展性更好**：轻松支持流式、并行、条件分支、回退等高级功能
- **类型更安全**：LangChain 自动处理组件间的输入输出转换

简单来说，**LCEL 让构建 LLM 应用从“写代码”变成了“组装流程”**，这也是目前 LangChain 推荐的主要开发方式。

2.4.3 LCEL 的第一个例子

下面我们通过一个完整的小例子来感受 LCEL 的魅力：

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# 1. 定义 Prompt
prompt = ChatPromptTemplate.from_template("请用一句话解释：{concept}")

# 2. 定义模型
llm = ChatOpenAI(model="deepseek-chat")

# 3. 定义输出解析器
```

```python
parser = StrOutputParser()

# 4. 使用 LCEL 组合成链
chain = prompt | llm | parser

# 5. 调用
result = chain.invoke({"concept": "量子纠缠"})
print(result)
```

输出示例：

```
量子纠缠是指两个或多个粒子即使相隔很远，它们的状态仍然相互关联的现象。
```

在这个例子中，我们清晰地看到了 LCEL 的三个核心组件：

- **Prompt**：负责格式化输入
- **LLM**：负责生成内容
- **Parser**：负责将输出整理成字符串

2.5 Runnable 的运行方式

前面我们已经知道，LCEL 可以通过管道符 `|` 将 Prompt、LLM、Output Parser 等组件组合成一条完整的 Chain：

```python
chain = prompt | llm | parser
```

但是，Chain 构建完成之后，并不只有一种运行方式。

由于 LCEL 底层遵循 Runnable 协议，因此只要一个对象是 Runnable，它通常就可以使用一组统一的运行方法。也就是说，不管这个 Runnable 是一个单独的模型、一个提示模板，还是一条完整的 LCEL Chain，我们都可以用类似的方式调用它。

常见运行方式如下：

方法	作用	典型场景
`invoke`	单次调用	处理一个输入
`batch`	批量调用	一次处理多个输入
`stream`	流式输出	边生成边返回
`ainvoke`	异步单次调用	异步程序中处理一个输入
`abatch`	异步批量调用	异步程序中批量处理
`astream`	异步流式输出	异步程序中边生成边返回

本节重点掌握前三个方法：

```
invoke  →  单次调用
batch   →  批量调用
stream  →  流式输出
```

异步方法通常用于 Web 服务、高并发接口或异步任务框架中，前期只需要知道它们的作用即可。

2.5.1 invoke：单次调用

`invoke` 是 Runnable 最基础、最常用的运行方法，它的作用是：给 Runnable 一个输入，得到一个输出。

例如，前面我们已经构建了一条 Chain：

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("请用一句话解释：{concept}")
llm = ChatOpenAI(model="deepseek-chat")
parser = StrOutputParser()

chain = prompt | llm | parser
```

此时可以使用 `invoke` 进行单次调用：

```python
result = chain.invoke({
    "concept": "量子纠缠"
})

print(result)
```

输出示例：

```
量子纠缠是指两个或多个粒子即使相距很远，它们的状态仍然会保持某种关联的现象。
```

在这个过程中，数据会按照 LCEL 的管道顺序依次流动：

```
dict 输入
   ↓
PromptTemplate
   ↓
PromptValue
   ↓
ChatOpenAI
   ↓
AIMessage
   ↓
StrOutputParser
   ↓
str 输出
```

也就是说，虽然我们只调用了一次 `chain.invoke(...)`，但实际上，整条链中的每个组件都会依次执行。这也是 LCEL 的核心优势之一：开发者只需要关注整体流程，不需要手动管理每一步之间的数据传递。

2.5.2 batch：批量调用

`batch` 用于一次性处理多个输入，如果我们有多个概念需要让模型解释，最直接的做法可能是写一个循环：

```
concepts = ["量子纠缠", "机器学习", "大语言模型"]

for concept in concepts:
    result = chain.invoke({"concept": concept})
    print(result)
```

这种写法当然可以运行，但它没有充分利用 Runnable 协议提供的批量能力，更推荐的方式是使用 `batch`：

```
results = chain.batch([
    {"concept": "量子纠缠"},
    {"concept": "机器学习"},
    {"concept": "大语言模型"}
])

for result in results:
    print(result)
```

`batch` 的输入是一个列表，列表中的每个元素都是一次独立调用的输入。

因此：`chain.invoke({"concept": "量子纠缠"})`表示处理一个输入。

而：

```
chain.batch([
    {"concept": "量子纠缠"},
    {"concept": "机器学习"},
    {"concept": "大语言模型"}
])
```

表示一次处理多个输入。

输出结果也是一个列表：

```
[
    "量子纠缠是指……",
    "机器学习是指……",
    "大语言模型是指……"
]
```

2.5.3 stream：流式输出

`stream` 用于流式输出，也就是模型一边生成内容，一边把结果返回给程序。在普通的 `invoke` 调用中，程序会等待模型完整生成结束后，才一次性拿到最终结果：

```
result = chain.invoke({
    "concept": "大语言模型"
})

print(result)
```

这种方式适合普通任务，但在聊天机器人场景中，用户体验并不好，如果模型需要生成一大段内容，用户就必须等待整段内容全部生成完之后才能看到结果。

而流式输出的方式是：

```
for chunk in chain.stream({
    "concept": "大语言模型"
}):
    print(chunk, end="")
```

这段代码的含义是：

```
模型每生成一小段内容，就立刻返回一小段内容。
程序每收到一小段内容，就立刻打印出来。
```

这样用户看到的效果就是：

```
模型正在逐字、逐句地输出内容。
```

这也是很多聊天机器人能够实现“打字机效果”的原因。

需要注意的是，如果 Chain 的最后接了 `StrOutputParser`，那么 `stream` 返回的通常就是一段段字符串片段。

例如：

```
for chunk in chain.stream({"concept": "大语言模型"}):
    print(chunk, end="")
```

输出效果可能类似：

```
大语言模型是一种基于深度学习的人工智能模型，能够理解和生成自然语言文本。
```

虽然最终看起来是一整句话，但程序实际上是分多次接收到这些内容的。

可以简单理解为：

```
invoke  →  等全部生成完，再一次性返回
stream  →  边生成，边返回
```

2.5.4 异步运行方法

除了同步方法之外，Runnable 还提供了一组异步运行方法：

同步方法	异步方法	说明
`invoke`	`ainvoke`	异步单次调用
`batch`	`abatch`	异步批量调用
`stream`	`astream`	异步流式输出

异步方法通常需要配合 Python 的 `async` 和 `await` 使用。

例如：

```
result = await chain.ainvoke({
    "concept": "大语言模型"
})
```

异步流式输出则可以写成：

```
async for chunk in chain.astream({
    "concept": "大语言模型"
}):
    print(chunk, end="")
```

在前期学习中，不需要深入展开异步编程。只需要知道：

```
同步方法适合普通脚本和基础项目。
异步方法适合 web 服务、高并发接口和异步任务系统。
```

例如，在 FastAPI 这类异步 Web 框架中，如果后端需要同时处理多个用户请求，就经常会使用异步调用方式。

2.5.5 小结

Runnable 的运行方式可以总结为三组：

```
单次调用：invoke / ainvoke
批量调用：batch / abatch
流式输出：stream / astream
```

其中，前期最重要的是：

```
invoke：跑一次
batch：跑一批
stream：边生成边返回
```

Runnable 协议的价值不仅在于统一了组件的输入输出，也在于统一了组件的运行方式。无论是一个单独的模型，还是一条由 LCEL 拼接出来的复杂 Chain，只要它是 Runnable，就可以使用统一的方法进行调用。这意味着 LCEL 不只是让代码写起来更简洁，更重要的是，它让 LLM 应用具备了更强的工程能力。

因此，LCEL 的本质不是简单的“管道符语法”，而是一套围绕 Runnable 协议构建的工程化组合方式。